

10-Day Onboarding & Training Plan

MERN + MySQL + Next.js Track — Intern: Pratik Bagul

React.js

Node.js

MySQL

Next.js

Intern	Pratik Bagul — MERN Stack Developer (Fresher)
Existing Strengths	React.js, Node.js/Express, MongoDB, JWT Auth, REST APIs, Git
Skill Gaps to Close	MySQL (relational DB), Next.js (SSR/App Router framework)
Program Duration	10 Working Days
Training Format	Guided theory (mornings) + hands-on project tasks (afternoons)
Prepared For	Trainer / Team Lead Use

Program Overview

This plan is designed around a key observation: the intern already has solid, project-tested fundamentals in **React.js** and **Node.js/Express**, including REST API design, JWT authentication, and CRUD operations with MongoDB. The training does not need to teach these from scratch — it needs to **bridge his existing MERN knowledge into the MySQL + Next.js stack** used by the team.

The two real gaps are:

- **MySQL** — he has only worked with MongoDB (schema-less, NoSQL). Relational concepts — tables, foreign keys, joins, normalization — will be genuinely new and need dedicated time (Days 3–4).
- **Next.js** — he has only used plain React (component-based SPA). Server-side rendering, file-based routing, and API routes are new mental models (Days 1–2, 5).

The structure follows a **"learn → apply → integrate"** rhythm: each new concept is taught in the morning and immediately applied to a hands-on task in the afternoon, culminating in one integrated capstone project across the final four days.

10-Day Snapshot

Day	Focus Area	Key Concepts	Deliverable
1	Next.js Foundations	App Router, file-based routing, layouts, navigation	3-page static Next.js site
2	Rendering & Data Fetching	Server vs client components, loading/error states	MERN list page ported to Next.js
3	MySQL Fundamentals	Tables, keys, relationships, joins, normalization	Designed & created SQL schema
4	Node.js + MySQL	Prisma ORM, migrations, CRUD queries	UserList API rebuilt on MySQL
5	Next.js API Routes	Route handlers, server-side DB calls	Self-contained Next.js CRUD API
6	Auth in Next.js	JWT + cookies, middleware, protected routes	Working login/dashboard flow
7	Capstone — Build Day 1	Architecture planning, schema design, scaffolding	Task Manager app: core structure
8	Capstone — Build Day 2	Relational CRUD, filtering, pagination	Task Manager app: full functionality
9	Polish & Deployment	Validation, error handling, env config, hosting	Live deployed application
10	Review & Presentation	Code walkthrough, security review, feedback	Documented feedback + growth plan

Trainer note: Days 3–4 (MySQL) are the highest-risk block — the NoSQL-to-SQL mental shift is where most MERN developers slow down. Prisma is used instead of raw SQL queries in Node because its model syntax closely mirrors Mongoose, which accelerates the transition.

Next.js Foundations

Goal: Understand why Next.js exists and get comfortable navigating the App Router before touching data.

Next.js

App Router

Routing

React (recap)

Morning — Concept Session (2–3 hrs)

Explain the "why" before the "how" — he already understands React deeply, so anchor everything by contrasting it with what he knows from his CRA-style projects (UserList, Cars24, Pepperfry):

- Client-Side Rendering (what CRA/plain React does, which he's used) vs Server-Side Rendering / Static Generation (what Next.js adds)
- Why companies prefer Next.js: SEO, faster first load, built-in routing, built-in backend (API routes)
- Project setup: `npm create-next-app@latest` with App Router + TypeScript optional (recommend JS first since he's not used TS)
- Folder structure walkthrough: `app/`, `page.js`, `layout.js`, `public/`
- File-based routing: how a folder = a route, no more manually configuring React Router

HANDS-ON TASK

Build a 3-page Next.js site: Home / About / Contact.

Explanation of the task: This mirrors the simplest thing he already knows how to build in React (a multi-page site with navigation) but forces him to use Next.js's routing system instead of React Router. He should create `app/page.js` (Home), `app/about/page.js`, and `app/contact/page.js`, then a shared `layout.js` with a navbar using Next's `<Link>` component. This proves he understands that **folder structure defines the route** — a completely different mental model from the `<Route path="/about">` pattern he used in his Cars24/Pepperfry projects.

- Use a shared layout with header/footer across all 3 pages
- Use `<Link>` for navigation (not `<a>` tags) and explain why
- Add a simple contact form UI (no submission logic yet — just the interface)

End-of-day check: Can he explain, in his own words, the difference between a "page" in React Router and a "page" in Next.js App Router? Can he create a new route without help?

Rendering & Data Fetching in Next.js

Goal: Understand server vs client components and fetch real data the Next.js way.

[Server Components](#)[Client Components](#)[Data Fetching](#)

Morning — Concept Session

- Server Components (default in App Router) vs Client Components (`"use client"`) — when to use which
- Why `useState` / `useEffect` (which he's used heavily in UserList app) only work in Client Components
- Fetching data directly inside a Server Component with `async/await` — no more `useEffect` + `fetch` boilerplate for simple reads
- `loading.js` and `error.js` conventions for automatic loading/error UI

HANDS-ON TASK

Port the "UserList" project's listing page into Next.js.

Explanation of the task: He already built a working UserList CRUD app in React using `useState` / `useEffect` to fetch and display users from his Express API. The task is to rebuild *just the listing page* as a Next.js Server Component that fetches the same data directly with `async/await` inside the component — no loading spinner state needed, since Next.js handles it via `loading.js`. This is the clearest possible way to show him the before/after: same data, same API, fundamentally different fetching pattern.

- Keep his existing Express + MongoDB API running as the data source for now
- Build the list page as a Server Component (no `"use client"`)
- Add a `loading.js` skeleton screen and an `error.js` fallback
- Add one small interactive piece (e.g. a search/filter input) as a separate Client Component to practice the split

End-of-day check: Given a new component, can he correctly decide whether it needs to be a Server or Client Component, and explain why?

MySQL Fundamentals

Goal: Build a relational mental model from scratch — this is the biggest conceptual shift in the program.

SQL

Relational Design

Normalization

Morning — Concept Session (spend extra time here)

Contrast everything against MongoDB, since that's his reference point from the UserList and Student Dashboard projects:

- Documents/collections (MongoDB) vs tables/rows (MySQL) — fixed schema vs flexible schema
- Primary keys, foreign keys, and how relationships replace MongoDB's embedded/referenced documents
- Relationship types: one-to-one, one-to-many, many-to-many (with junction tables)
- Normalization basics (1NF/2NF/3NF) — why you don't just dump everything in one table
- Core SQL syntax: `CREATE TABLE`, `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, `ORDER BY`

HANDS-ON TASK

Design and create a schema for a "Users + Posts" mini blog system.

Explanation of the task: Rather than jumping straight into code, this task forces him to think in tables before he writes a single query — the skill most MongoDB-first developers skip and later struggle with. He should design two related tables (`users` and `posts` , where a post belongs to a user via a foreign key), install MySQL locally (or use a Docker container), and write the raw `CREATE TABLE` statements himself. Then he practices querying: insert sample data, and write `SELECT` queries that `JOIN` the two tables (e.g. "get all posts with their author's name").

- Draw the schema on paper/whiteboard first (or a free tool like dbdiagram.io) before writing SQL
- Write `CREATE TABLE` statements with proper primary/foreign keys
- Insert at least 5 users and 10 posts as sample data
- Write 5 `SELECT` queries of increasing complexity, ending with a `JOIN` + `GROUP BY` (e.g. "count of posts per user")

End-of-day check: Can he explain why you can't just "embed" posts inside a user row the way he would in MongoDB, and what problem foreign keys solve instead? Don't move to Day 4 until this clicks.

Connecting Node.js to MySQL

Goal: Rebuild a familiar API — but on MySQL instead of MongoDB — using an ORM he'll find familiar.

Prisma ORM

Express.js

CRUD

Morning — Concept Session

- Why use an ORM instead of raw SQL strings in application code (safety, readability, migrations)
- Introduce Prisma — deliberately chosen because its `schema.prisma` model syntax and query methods (`findMany` , `create` , `update`) closely resemble Mongoose, which he already knows well
- Prisma migrations: how schema changes become tracked SQL migration files
- Connecting Prisma Client inside Express route handlers

HANDS-ON TASK

Rebuild the "UserList" CRUD API on MySQL instead of MongoDB.

Explanation of the task: This is a direct, deliberate parallel to his existing project — same functionality (create/read/update/delete users), same Express.js server structure, only the data layer changes. Because the business logic (routes, controllers) stays nearly identical, this isolates exactly what's different about working with MySQL vs MongoDB, which reinforces Day 3's concepts through repetition rather than new theory.

- Define a `User` model in `schema.prisma` and run a migration to create the table
- Rebuild the 5 REST endpoints from his original project (GET all, GET one, POST, PUT, DELETE) using Prisma Client
- Add one relational field this time (e.g. each user has many "notes") to practice a one-to-many relationship end-to-end
- Test all endpoints in Postman, same as his original project

End-of-day check: Can he write a Prisma query for a one-to-many relationship (e.g. "get a user with all their notes") without referring to docs?

Next.js API Routes

Goal: Learn to build backend logic inside Next.js itself, removing the need for a separate Express server.

Route Handlers

Full-stack Next.js

MySQL

Morning — Concept Session

- Route Handlers: `app/api/<route>/route.js` exporting `GET` / `POST` / `PUT` / `DELETE` functions
- How this replaces a standalone Express server — frontend and backend in one project
- Reusing the same Prisma Client setup from Day 4 inside Next.js API routes
- Request/response handling differences from Express (`NextResponse` vs `res.json()`)

HANDS-ON TASK

Convert the Day 4 API into native Next.js API routes, and connect the Day 1–2 frontend to it.

Explanation of the task: This is the "click" moment where the two new skills (Next.js + MySQL) merge into one working full-stack app. He takes the Prisma-based CRUD logic he wrote in plain Express on Day 4 and moves it into Next.js Route Handlers, then wires his Day 2 listing page to call these new endpoints instead of the old Express server. By the end of today he has a single Next.js project doing everything his separate React + Express + MongoDB UserList project did — but on the new stack.

- Create `app/api/users/route.js` (GET all, POST) and `app/api/users/[id]/route.js` (GET one, PUT, DELETE)
- Update the frontend list/detail pages to call these new local API routes
- Add a simple "Add User" form as a Client Component that POSTs to the API route
- Confirm full CRUD works end-to-end inside one Next.js project

End-of-day check: He should be able to describe, unprompted, why a Next.js API route replaces the need for a separate Express server in this kind of project.

Authentication in Next.js

Goal: Adapt his existing JWT knowledge to Next.js's request/routing model, including protected routes.

JWT

Cookies

Middleware

Morning — Concept Session

- Recap his existing JWT knowledge from the Student Dashboard project (login → issue token → verify on protected routes)
- Where to store the token in Next.js: httpOnly cookies vs localStorage, and why cookies are safer for SSR apps
- Next.js Middleware (`middleware.js`) for protecting routes before a page even renders
- Reading auth state in Server Components vs Client Components

HANDS-ON TASK

Add login/signup and a protected dashboard route to the project.

Explanation of the task: This mirrors the secure login system he already built in his Student Dashboard Application — but instead of managing the token in React state/localStorage, he'll issue it from a Next.js API route, set it as an httpOnly cookie, and use Next.js Middleware to block access to `/dashboard` for anyone without a valid token. This teaches him that "protecting a route" in Next.js happens at the framework level, not just by conditionally rendering JSX like he did before.

- Build `/api/auth/signup` and `/api/auth/login` routes (hash passwords, issue JWT)
- Set the JWT as an httpOnly cookie on login
- Create `middleware.js` that redirects unauthenticated users away from `/dashboard`
- Build a simple protected `/dashboard` page showing the logged-in user's data

End-of-day check: Can he explain the security reasoning for httpOnly cookies over localStorage, given he previously used localStorage-style patterns?

Task Manager App: Planning & Core Structure

Goal: Independently plan and scaffold a full integrated project using everything learned so far.

Capstone Project

Schema Design

Architecture

Format Change from Today

From Day 7 onward, the trainer shifts from teaching to **reviewing**. The intern should be building independently, with the trainer available for questions and doing a short architecture review before coding begins each day.

CAPSTONE TASK — ASSIGNED PROJECT: "TASK MANAGER"

Explanation of the project: A Task Manager is deliberately chosen because it requires everything covered in the previous six days in one coherent app: a relational schema (users → tasks, one-to-many), Next.js pages and API routes, authentication, and CRUD — the same shape as his own UserList and Student Dashboard projects, but on the new stack, done with less hand-holding.

Today's specific deliverables:

- Design the schema on paper first: **User** and **Task** tables (task has status, due date, priority, and belongs to a user)
- Get schema signed off by the trainer before writing any Prisma code (this is the checkpoint)
- Scaffold the Next.js project structure: routes for **/login** , **/dashboard** , **/tasks**
- Set up Prisma models and run the initial migration
- Reuse his Day 6 auth flow as the starting point for this new project

Trainer checkpoint: Review the schema design before he writes code. Look specifically for: correct foreign key usage, sensible field types, and whether he's thinking in relations rather than trying to nest data MongoDB-style.

Task Manager App: Full CRUD & Relational Features

Goal: Complete core functionality, including features that specifically exercise relational SQL skills.

CRUD

Filtering

Pagination

CAPSTONE TASK — CONTINUED

Explanation of the task: Today's work is intentionally chosen to go slightly beyond simple CRUD (which he's already proven he can do in his prior projects) and into query patterns that only make sense in a relational database — filtering by relation, sorting, and pagination via SQL rather than in-memory JavaScript array methods, which is what he'd likely default to from his React experience.

- Full CRUD for tasks: create, edit, mark complete, delete — scoped to the logged-in user only
- Filter tasks by status (pending/completed) and priority using SQL `WHERE` clauses via Prisma, not client-side filtering
- Add pagination on the task list using Prisma's `skip / take`
- Add a dashboard summary widget: "X tasks pending, Y completed" using a Prisma aggregate/count query
- Basic UI polish using Tailwind CSS (already in his skill set)

Trainer checkpoint: Check whether he pushed filtering/pagination logic down to the database query (correct) versus fetching everything and filtering in JavaScript (common mistake for developers coming from MongoDB/array-heavy thinking).

Polish, Validation & Deployment

Goal: Take the project from "working on my machine" to a production-ready, deployed application.

Validation

Error Handling

Deployment

Morning — Concept Session

- Input validation on API routes (e.g. using Zod) — never trust client input
- Centralized error handling and meaningful HTTP status codes
- Environment variables: `.env` usage for DB connection strings and JWT secrets, and why they're never committed to Git
- Deployment overview: Vercel for the Next.js app, a hosted MySQL provider (e.g. PlanetScale or Railway) for the database

HANDS-ON TASK

Harden and deploy the Task Manager application.

Explanation of the task: Deployment is a skill his resume shows no evidence of yet — all his listed projects appear to be local/portfolio builds. This task closes that gap and also reinforces production discipline: validating input, not leaking stack traces to the client, and correctly separating secrets from code, all of which matter far more on a real team than in solo practice projects.

- Add validation to all API routes (required fields, correct types, sane limits)
- Return consistent error response shapes (status code + message) instead of raw errors
- Move DB URL and JWT secret into `.env`, confirm `.env` is git-ignored
- Provision a free-tier hosted MySQL database and point Prisma at it
- Deploy the app to Vercel and confirm the live URL works end-to-end (signup → login → create task)

End-of-day check: A live, shareable URL where the trainer can sign up, log in, and manage tasks without any local setup.

Code Review, Presentation & Feedback

Goal: Consolidate learning, surface gaps, and set a concrete growth plan going forward.

[Code Review](#)[Security](#)[Feedback](#)

Session Structure

1. **Intern-led walkthrough (20–30 min):** He presents the Task Manager app — architecture decisions, schema design, and any challenges he hit switching from MongoDB/plain React to MySQL/Next.js.
2. **Trainer code review (30–40 min):** Go through the codebase together, focusing on the areas below.
3. **Written feedback:** Document strengths, gaps, and a concrete next-30-days learning plan.

Review Focus Areas

Area	What to check
SQL query quality	Are queries using proper indexing, filtering at the DB level, avoiding N+1 query problems from Prisma calls in loops?
Schema design	Correct normalization, sensible foreign keys, appropriate field types and constraints
Next.js architecture	Correct Server/Client Component split, proper use of API routes vs direct DB calls in Server Components
Security	SQL injection protection (should be automatic via Prisma), password hashing, JWT stored securely, no secrets in code
Code structure	Consistent with the clean, modular habits already visible in his Cars24/Pepperfry projects
Error handling	Graceful failures, meaningful messages, no unhandled promise rejections

Deliverable of the Day

- ☐ Written feedback document: 3 strengths, 2–3 growth areas
- ☐ A next-30-days plan (e.g. TypeScript adoption, testing with Jest, more advanced Next.js features like ISR/Server Actions)
- ☐ Confirmation of role-readiness or identification of any area needing a short follow-up session

Success signal for the whole program: By Day 7, he should be building with minimal guidance and only needing checkpoint reviews — not active teaching. If that shift didn't happen by Day 8, it's worth an extra day on MySQL/Prisma fundamentals before moving further.